

SKILL WEEK – 6

TASK 1: To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

```
shreyaj@LAPTOP-FT6VVM4D: ~$ mkdir -p ~/Week6
cd ~/Week6
shreyaj@LAPTOP-FT6VVM4D:~/Week6$ nano session11_task1.c
shreyaj@LAPTOP-FT6VVM4D:~/Week6$ gcc session11_task1.c -o session11_task1
./session11_task1
Session 10 - Command History
Enter commands (type 'history' to display, 'exit' to quit).

mysHELL> ls
pwd
echo hello
history
exitCommand stored in history.
mysHELL> Command stored in history.
mysHELL> Command stored in history.
mysHELL>
Command History:
1 ls
2 pwd
3 echo hello

History validation: SUCCESS
mysHELL> ls
Command stored in history.
mysHELL> pwd
Command stored in history.
mysHELL> echo hello
Command stored in history.
mysHELL> history
Command History:
1 ls
2 pwd
3 echo hello
4 exitls
5 pwd
6 echo hello

History validation: SUCCESS
mysHELL> exit

Exiting shell...

History buffer contains 6 command(s).
History consistency verified.
shreyaj@LAPTOP-FT6VVM4D:~/Week6$ |
```

```
GNU nano 8.7.1 session11_task1.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_HISTORY 10
#define MAX_COMMAND 200

int main() {
    char history[MAX_HISTORY][MAX_COMMAND];
    char input[MAX_COMMAND];
    int count = 0;
    int next = 0;

    printf("Session 10 - Command History\n");
    printf("Enter commands (type 'history' to display, 'exit' to quit).\n\n");

    while (1) {
        printf("mysHELL> ");
        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        /* Ignore empty commands */
        if (strlen(input) == 0)
            continue;

        /* Display history */
        if (strcmp(input, "history") == 0) {
            printf("\nCommand History:\n");
            if (count == 0) {
                printf("History is empty.\n");
            } else {
                int start = (count < MAX_HISTORY) ? 0 : next;
                for (int i = 0; i < count; i++) {
                    int index = (start + i) % MAX_HISTORY;
                    printf("%d %s\n", i + 1, history[index]);
                }
                printf("\nHistory validation: SUCCESS\n");
            }
            continue;
        }

        if (strcmp(input, "exit") == 0) {
            return 0;
        }

        if (count < MAX_HISTORY) {
            strcpy(history[count], input);
            count++;
        } else {
            next = next + 1;
        }
    }
}
```

```
shreyaj@LAPTOP-FT6VVM4D: x + v session11_task1.c
GNU nano 8.7.1
input[strespn(input, "\n")] = '\0';

/* Ignore empty commands */
if (strlen(input) == 0)
    continue;

/* Display history */
if (strcmp(input, "history") == 0) {
    printf("\nCommand History:\n");
    if (count == 0) {
        printf("History is empty.\n");
    } else {
        int start = (count < MAX_HISTORY) ? 0 : next;
        for (int i = 0; i < count; i++) {
            int index = (start + i) % MAX_HISTORY;
            printf("%d %s\n", i + 1, history[index]);
        }
        printf("\nHistory validation: SUCCESS\n");
    }
    continue;
}

/* Exit */
if (strcmp(input, "exit") == 0) {
    printf("\nExiting shell...\n");
    break;
}

/* Store command */
strcpy(history[next], input);
next = (next + 1) % MAX_HISTORY;
if (count < MAX_HISTORY)
    count++;

printf("Command stored in history.\n");

printf("\nHistory buffer contains %d command(s).\n", count);
printf("History consistency verified.\n");
return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   ^U Undo       ^- Set Mark   ^_ To Bracket
^X Exit      ^R Read File  ^N Replace    ^P Paste      ^J Justify    ^_ Go To Line ^E Redo       ^M Copy       ^B Where Was
```

TASK 2: To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

```
shreyaj@LAPTOP-FT6VVM4D:~/Week6$ nano session11_task2.c
shreyaj@LAPTOP-FT6VVM4D:~/Week6$ gcc session11_task2.c -o session11_task2
./session11_task2
Session 10 - Pipeline Execution
Enter a pipeline command:
myshell> ls | wc -l

Pipeline Structure:
Command 1: ls

Pipeline validation: SUCCESS
Execution order maintained.
session11_task1 session11_task1.c session11_task2 session11_task2.c

All processes completed.
All pipe resources cleaned up.
Long pipeline executed successfully.
```

```
GNU nano 8.7.1 session11_task2.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_COMMANDS 10
#define MAX_ARGS 20
#define MAX_INPUT 500

typedef struct {
    char *args[MAX_ARGS];
    int argc;
} Command;

int parse_pipeline(char *input, Command commands[]) {
    int count = 0;
    char *part = strtok(input, "|");

    while (part != NULL && count < MAX_COMMANDS) {
        commands[count].argc = 0;
        char *arg = strtok(part, " \\t\\n");
        while (arg != NULL && commands[count].argc < MAX_ARGS - 1) {
            commands[count].args[commands[count].argc++] = arg;
            arg = strtok(NULL, " \\t\\n");
        }
        commands[count].args[commands[count].argc] = NULL;

        if (commands[count].argc == 0) {
            printf("Error: Empty command in pipeline.\\n");
            return -1;
        }
        count++;
        part = strtok(NULL, "|");
    }
    return count;
}

int main() {
    char input[MAX_INPUT];
    Command commands[MAX_COMMANDS];
```

```
GNU nano 8.7.1 session11_task2.c
int main() {
    char input[MAX_INPUT];
    Command commands[MAX_COMMANDS];

    printf("Session 10 - Pipeline Execution\\n");
    printf("Enter a pipeline command:\\n");
    printf("myshell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\\n")] = '\\0';

    if (strlen(input) == 0) {
        printf("Empty pipeline.\\n");
        return 0;
    }

    int command_count = parse_pipeline(input, commands);
    if (command_count <= 0) {
        printf("Pipeline validation failed.\\n");
        return 1;
    }

    printf("\\n\\nPipeline Structure:\\n");
    for (int i = 0; i < command_count; i++) {
        printf("Command %d: ", i + 1);
        for (int j = 0; j < commands[i].argc; j++) {
            printf("%s ", commands[i].args[j]);
        }
        printf("\\n");
    }

    printf("\\n\\nPipeline validation: SUCCESS\\n");
    printf("Execution order maintained.\\n");

    int pipes[MAX_COMMANDS - 1][2];
    pid_t pids[MAX_COMMANDS];

    for (int i = 0; i < command_count - 1; i++) {
        if (pipe(pipes[i]) == -1) {
            perror("pipe failed");
            return 1;
        }
    }
```

```
shreyaj@LAPTOP-FT6VVM4D: x + v session11_task2.c
GNU nano 8.7.1
for (int i = 0; i < command_count; i++) {
    pids[i] = fork();
    if (pids[i] < 0) {
        perror("fork failed");
        return 1;
    }

    if (pids[i] == 0) {
        /* Child: connect input from previous pipe */
        if (i > 0) {
            dup2(pipes[i - 1][0], STDIN_FILENO);
        }
        /* Child: connect output to next pipe */
        if (i < command_count - 1) {
            dup2(pipes[i][1], STDOUT_FILENO);
        }
        /* Close all pipe descriptors in child */
        for (int j = 0; j < command_count - 1; j++) {
            close(pipes[j][0]);
            close(pipes[j][1]);
        }
        execvp(commands[i].args[0], commands[i].args);
        perror("execvp failed");
        exit(EXIT_FAILURE);
    }
}

/* Parent closes all pipe descriptors */
for (int i = 0; i < command_count - 1; i++) {
    close(pipes[i][0]);
    close(pipes[i][1]);
}

/* Coordinate execution */
for (int i = 0; i < command_count; i++) {
    waitpid(pids[i], NULL, 0);
}

printf("\nAll processes completed.\n");
printf("All pipe resources cleaned up.\n");
printf("Long pipeline executed successfully.\n");
return 0;
}
```

Session – 12

TASK – 1: To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow

```
shreyaj@LAPTOP-FT6VVM4D:~/Week6$ nano session12_task1.c
shreyaj@LAPTOP-FT6VVM4D:~/Week6$ gcc session12_task1.c -o session12_task1
./session12_task1
Session 10 - Pipe and Stream Redirection
Pipe created successfully.
Connected commands executing...
4
Data flow completed successfully.
Parent process synchronized with both children.
shreyaj@LAPTOP-FT6VVM4D:~/Week6$ |
```

```
shreyaj@LAPTOP-FT6VVM4D: x + v session12_task1.c
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    int pipefd[2];
    pid_t child1, child2;

    if (pipe(pipefd) == -1) {
        perror("pipe failed");
        return 1;
    }

    printf("Session 10 - Pipe and Stream Redirection\n");
    printf("Pipe created successfully.\n");

    /* Create first child */
    child1 = fork();
    if (child1 < 0) {
        perror("fork failed");
        return 1;
    }

    if (child1 == 0) {
        /* First child writes to pipe */
        close(pipefd[0]);
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);
        execlp("echo", "echo", "Hello from child process", NULL);
        perror("exec failed");
        exit(EXIT_FAILURE);
    }

    /* Create second child */
    child2 = fork();
    if (child2 < 0) {
        perror("fork failed");
        return 1;
    }

    if (child2 == 0) {
```

```
shreyaj@LAPTOP-FT6VVM4D: x + v session12_task1.c
GNU nano 8.7.1

    if (child1 == 0) {
        /* First child writes to pipe */
        close(pipefd[0]);
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);
        execlp("echo", "echo", "Hello from child process", NULL);
        perror("exec failed");
        exit(EXIT_FAILURE);
    }

    /* Create second child */
    child2 = fork();
    if (child2 < 0) {
        perror("fork failed");
        return 1;
    }

    if (child2 == 0) {
        /* Second child reads from pipe */
        close(pipefd[1]);
        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);
        execlp("wc", "wc", "-w", NULL);
        perror("exec failed");
        exit(EXIT_FAILURE);
    }

    /* Parent closes both pipe descriptors */
    close(pipefd[0]);
    close(pipefd[1]);

    printf("Connected commands executing...\n");

    /* Synchronize child processes */
    waitpid(child1, NULL, 0);
    waitpid(child2, NULL, 0);

    printf("Data flow completed successfully.\n");
    printf("Parent process synchronized with both children.\n");
    return 0;
}
```

TASK – 2: To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines

```

shreyaj@LAPTOP-FT6VVM4D:~/Week6$ nano session12_task2.c
shreyaj@LAPTOP-FT6VVM4D:~/Week6$ gcc session12_task2.c -o session12_task2
./session12_task2
Session 12 - Multiple Pipe Execution
Enter a long pipeline:
myshell> ls | grep c | wc -l

Pipeline Structure:
Command 1: ls
Command 2: grep c
Command 3: wc -l

Pipeline validation: SUCCESS
Execution order maintained.
4

All processes completed.
All pipe resources cleaned up.
Long pipeline executed successfully.
shreyaj@LAPTOP-FT6VVM4D:~/Week6$

```

```

GNU nano 8.7.1 session12_task2.c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_CMDS 10
#define MAX_ARGS 20
#define MAX_LINE 500

typedef struct {
    char *argv[MAX_ARGS];
    int argc;
} Cmd;

int parse_pipeline(char *line, Cmd cmds[]) {
    int n = 0;
    char *saveptr1;
    char *part = strtok_r(line, "|", &saveptr1);

    while (part != NULL && n < MAX_CMDS) {
        cmds[n].argc = 0;
        char *saveptr2;
        char *arg = strtok_r(part, "\\t\\n", &saveptr2);

        while (arg != NULL && cmds[n].argc < MAX_ARGS - 1) {
            cmds[n].argv[cmds[n].argc++] = arg;
            arg = strtok_r(NULL, "\\t\\n", &saveptr2);
        }
        cmds[n].argv[cmds[n].argc] = NULL;

        if (cmds[n].argc == 0)
            return -1;

        n++;
        part = strtok_r(NULL, "|", &saveptr1);
    }
    return n;
}

int main(void) {

```

```

GNU nano 8.7.1 session12_task2.c
int main(void) {
    char line[MAX_LINE];
    Cmd cmds[MAX_CMDS];

    printf("Session 12 - Multiple Pipe Execution\\n");
    printf("Enter a long pipeline:\\n");
    printf("myshell> ");

    if (fgets(line, sizeof(line), stdin) == NULL)
        return 1;

    line[strcspn(line, "\\n")] = '\\0';

    if (strlen(line) == 0) {
        printf("Empty pipeline.\\n");
        return 0;
    }

    int count = parse_pipeline(line, cmds);
    if (count <= 0) {
        printf("Pipeline validation failed.\\n");
        return 1;
    }

    printf("\\nPipeline Structure:\\n");
    for (int i = 0; i < count; i++) {
        printf("Command %d: ", i + 1);
        for (int j = 0; j < cmds[i].argc; j++) {
            printf("%s ", cmds[i].argv[j]);
        }
        printf("\\n");
    }

    printf("\\nPipeline validation: SUCCESS\\n");
    printf("Execution order maintained.\\n");

    int pipes[MAX_CMDS - 1][2];
    pid_t pids[MAX_CMDS];

    // Create pipes
    for (int i = 0; i < count - 1; i++) {
        if (pipe(pipes[i]) == -1) {
            perror("pipe");

```

```
shreyaj@LAPTOP-FT6VVM4D: x + v session12_task2.c
GNU nano 8.7.1
// Create pipes
for (int i = 0; i < count - 1; i++) {
    if (pipe(pipes[i]) == -1) {
        perror("pipe");
        return 1;
    }
}

// Fork children
for (int i = 0; i < count; i++) {
    pids[i] = fork();
    if (pids[i] < 0) {
        perror("fork");
        return 1;
    }

    if (pids[i] == 0) {
        // Child i
        if (i > 0) {
            dup2(pipes[i - 1][0], STDIN_FILENO);
        }
        if (i < count - 1) {
            dup2(pipes[i][1], STDOUT_FILENO);
        }

        // Close all pipe fds in child
        for (int j = 0; j < count - 1; j++) {
            close(pipes[j][0]);
            close(pipes[j][1]);
        }

        execvp(cmds[i].argv[0], cmds[i].argv);
        perror("execvp");
        exit(EXIT_FAILURE);
    }
}

// Parent closes all pipe fds
for (int i = 0; i < count - 1; i++) {
    close(pipes[i][0]);
    close(pipes[i][1]);
}
```

```
}

// Parent closes all pipe fds
for (int i = 0; i < count - 1; i++) {
    close(pipes[i][0]);
    close(pipes[i][1]);
}

// Wait for all children
for (int i = 0; i < count; i++) {
    waitpid(pids[i], NULL, 0);
}

printf("\nAll processes completed.\n");
printf("All pipe resources cleaned up.\n");
printf("Long pipeline executed successfully.\n");
return 0;
}
```